

```
$ system --init db-design
```

CUSTOMER AND ORDER MANAGEMENT_SYSTEM

Database Design Document

tags: relational-db · schema · sql · reporting

```
$ cat table_of_contents.md
```

- 1. Introduction
- 2. Objectives
- 3. Entity-Relationship Overview
- 4. Table Design
- 5. Primary Key and Foreign Key Relationships
- 6. SQL Table Creation Scripts
- 7. Sample Data
- 8. Order Insertion, Updating, and Deletion
- 9. Customer-wise Order Reports
- 10. Sample Report Output
- 11. Conclusion

01 Introduction

The Customer and Order Management System is a relational database application designed to record customers and the orders they place. The system allows an organization to maintain customer information, associate each order with the customer who placed it, track order value and status, and generate customer-wise reports for business analysis. This document describes the database design, table structures, key relationships, and the SQL operations required to build and operate the system.

02 Objectives

- Design normalized tables for the Customer and Order entities.
- Establish primary key and foreign key relationships between the two tables.
- Store customer attributes such as name, email, and contact number.
- Store order attributes such as order date, amount, and status.
- Support insertion, updating, and deletion of order records.
- Generate customer-wise order reports for analysis and decision-making.

03 Entity-Relationship Overview

The system is built around two core entities: Customer and Order. A Customer can place many Orders, but each Order belongs to exactly one Customer. This is a one-to-many relationship, implemented using a foreign key (`customer_id`) in the Order table that references the primary key (`customer_id`) in the Customer table.

• • • *er-diagram.txt*

```
CUSTOMER (1) -----< (M) ORDER
  customer_id (PK)          order_id (PK)
                           customer_id (FK -> CUSTOMER.customer_id)
```

04 Table Design

// 4.1 customer table

Stores the list of registered customers.

Column Name	Data Type	Constraints	Description
customer_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for each customer
customer_name	VARCHAR(120)	NOT NULL	Full name of the customer
email	VARCHAR(150)	NOT NULL, UNIQUE	Contact email address
phone	VARCHAR(15)	NULL	Contact phone number
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Record creation date and time

// 4.2 order table

Stores individual order details, linked to a customer.

Column Name	Data Type	Constraints	Description
order_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for each order
customer_id	INT	FOREIGN KEY -> Customer(customer_id)	Links order to the placing customer
order_date	DATE	NOT NULL	Date the order was placed
amount	DECIMAL(10,2)	NOT NULL, CHECK (amount >= 0)	Total value of the order
status	VARCHAR(20)	NOT NULL, DEFAULT 'PENDING'	Order status (PENDING, SHIPPED, DELIVERED, CANCELLED)
updated_at	TIMESTAMP	ON UPDATE CURRENT_TIMESTAMP	Last update date and time

05 Primary Key and Foreign Key Relationships

► Customer.customer_id is the Primary Key of the Customer table.

- ▶ Order.order_id is the Primary Key of the Order table.
- ▶ Order.customer_id is a Foreign Key referencing Customer.customer_id, enforcing that every order must belong to a valid, existing customer.
- ▶ ON DELETE RESTRICT prevents a customer from being deleted while orders are still linked to them (can be changed to CASCADE if deleting a customer should delete their orders).

06 SQL Table Creation Scripts

• • • *create_customer.sql*

```
CREATE TABLE Customer (  
  customer_id INT AUTO_INCREMENT PRIMARY KEY,  
  customer_name VARCHAR(120) NOT NULL,  
  email VARCHAR(150) NOT NULL UNIQUE,  
  phone VARCHAR(15),  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

• • • *create_order.sql*

```
CREATE TABLE `Order` (  
  order_id INT AUTO_INCREMENT PRIMARY KEY,  
  customer_id INT NOT NULL,  
  order_date DATE NOT NULL,  
  amount DECIMAL(10,2) NOT NULL CHECK (amount >= 0),  
  status VARCHAR(20) NOT NULL DEFAULT 'PENDING',  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  CONSTRAINT fk_order_customer  
    FOREIGN KEY (customer_id) REFERENCES Customer(customer_id)  
    ON DELETE RESTRICT  
    ON UPDATE CASCADE  
);
```

07 Sample Data

// 7.1 insert sample customers

• • • *seed_customers.sql*

```
INSERT INTO Customer (customer_name, email, phone) VALUES  
( 'Ananya Rao', 'ananya.rao@example.com', '9840012345'),  
( 'Vikram Shah', 'vikram.shah@example.com', '9940098765'),  
( 'Priya Menon', 'priya.menon@example.com', '9003345678');
```

// 7.2 insert sample orders

• • • *seed_orders.sql*

```
INSERT INTO `Order` (customer_id, order_date, amount, status) VALUES  
(1, '2026-07-02', 2098.00, 'DELIVERED'),  
(1, '2026-07-20', 799.00, 'SHIPPED');
```

```
(2, '2026-07-15', 650.00, 'DELIVERED'),  
(3, '2026-08-01', 250.00, 'PENDING');
```

08 Order Insertion, Updating, and Deletion

// 8.1 insert a new order

```
• • • insert.sql
```

```
INSERT INTO `Order` (customer_id, order_date, amount, status)  
VALUES (2, '2026-08-10', 1499.00, 'PENDING');
```

// 8.2 update an order

```
• • • update.sql
```

```
-- Mark an order as shipped  
UPDATE `Order`  
SET status = 'SHIPPED'  
WHERE order_id = 4;
```

// 8.3 delete an order

```
• • • delete.sql
```

```
DELETE FROM `Order` WHERE order_id = 2;
```

09 Customer-wise Order Reports

// 9.1 list all orders under each customer

```
• • • report_all_orders.sql
```

```
SELECT c.customer_name, o.order_id, o.order_date, o.amount, o.status  
FROM Customer c  
JOIN `Order` o ON c.customer_id = o.customer_id  
ORDER BY c.customer_name, o.order_date;
```

// 9.2 customer-wise order count and total spend

```
• • • report_customer_spend.sql
```

```
SELECT c.customer_name,  
       COUNT(o.order_id) AS total_orders,  
       SUM(o.amount) AS total_spend  
FROM Customer c  
LEFT JOIN `Order` o ON c.customer_id = o.customer_id  
GROUP BY c.customer_name  
ORDER BY total_spend DESC;
```

// 9.3 pending orders report by customer

• • • *report_pending_orders.sql*

```
SELECT c.customer_name, o.order_id, o.order_date, o.amount
FROM Customer c
JOIN `Order` o ON c.customer_id = o.customer_id
WHERE o.status = 'PENDING'
ORDER BY c.customer_name;
```

10 Sample Report Output

Customer	Order ID	Amount (Rs.)	Status
Ananya Rao	1	2098.00	DELIVERED
Ananya Rao	2	799.00	SHIPPED
Vikram Shah	3	650.00	DELIVERED

11 Conclusion

This design provides a normalized, referentially-consistent structure for managing customers and orders. The one-to-many relationship between Customer and Order ensures data integrity, while the provided SQL scripts cover schema creation, CRUD operations, and customer-wise reporting needed for day-to-day sales and order-tracking management.

\$ process exited (code 0)